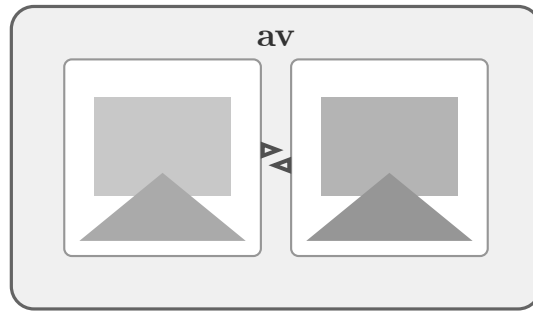




av

Alex's Image Viewer

Technical Specification

Version 0.1 2026-02-27

크로스 플랫폼 이미지 뷰어 / 비교 도구

차례

1. 개요	1
1.1. 프로젝트 목적	1
1.2. 설계 철학	1
1.3. 지원 플랫폼	1
1.4. 대상 사용자	1
2. 기술 스택	2
2.1. 선택 기준	2
2.2. 스택 요약	2
2.3. 의존성 상세	3
2.3.1. SDL3	3
2.3.2. Dear ImGui (docking branch)	3
2.3.3. stb_image / stb_image_write	3
2.3.4. lcms2	3
3. 아키텍처	4
3.1. 디렉토리 구조	4
3.2. 모듈 의존성 다이어그램	5
3.3. 데이터 흐름	5
3.4. 애플리케이션 상태 구조	6
4. 핵심 기능 사양	8
4.1. 이미지 로딩	8
4.1.1. 지원 포맷	8
4.1.2. 캐싱 정책	8
4.1.3. 색 공간 처리	8
4.2. 뷰포트 관리	8
4.2.1. 확대/축소 (Zoom)	8
4.2.2. 패닝 (Pan)	9
4.2.3. 뷰포트 동기화	9
4.3. 이미지 비교 모드	9
4.3.1. Side-by-Side 모드	9
4.3.2. Overlay (Swipe) 모드	9
4.3.3. Diff 모드	9
4.4. Diff 엔진	10
4.4.1. GPU Diff (실시간)	10
4.4.2. CPU Diff (SSIM)	10

4.4.3. Diff 모드 유형	10
4.5. 멀티 모니터 지원	11
4.6. 히스토그램	11
5. 키보드 바인딩	12
5.1. 파일 조작	12
5.2. 뷰포트 조작	12
5.3. 비교 모드 전환	13
5.4. Diff 파라미터	13
5.5. UI 패널 토글	13
6. CLI 인터페이스	14
6.1. 기본 사용법	14
6.2. 옵션 목록	14
6.3. 사용 예시	15
6.4. 종료 코드	15
7. 구현 로드맵	16
7.1. 단계별 계획	16
7.2. 상세 마일스톤	16
7.3. 빌드 환경 설정	16
7.3.1. 의존성 설치 (FetchContent)	16
7.3.2. 빌드 방법	17
7.4. 테스트 전략	17

1. 개요

1.1. 프로젝트 목적

av는 시각적으로 정밀한 이미지 비교와 빠른 탐색을 위해 설계된 크로스 플랫폼 이미지 뷰어이다. 렌더링 엔지니어, 아티스트, 연구자가 두 이미지의 미묘한 차이를 효율적으로 파악할 수 있도록 GPU 가속 diff 엔진과 직관적인 키보드 중심 인터페이스를 제공한다.

주요 사용 사례:

- 렌더링 결과물의 회귀 테스트 비교
- 머신러닝 모델의 출력 이미지 분석
- 색보정 전후 비교
- 아티스트 워크플로우에서의 빠른 이미지 리뷰

1.2. 설계 철학

av는 다음 원칙을 기반으로 설계된다.

원칙	설명
키보드 우선	마우스 없이 완전한 조작이 가능해야 한다. 모든 기능에 키 바인딩을 제공한다.
제로 레이턴시 diff	GPU fragment shader로 이미지 비교가 즉시 반응해야 한다.
경량 & 빠른 시작	수백 MB 프레임워크 없이 빠른 기동과 낮은 메모리 사용을 목표로 한다.
정확한 색 재현	ICC 색 프로파일 처리를 통해 색 공간을 정확하게 표시한다.
멀티 모니터 지원	OS 네이티브 윈도우로 여러 화면에 걸쳐 자유롭게 배치할 수 있다.
단순한 아키텍처	복잡한 의존성 없이 유지보수 용이한 구조를 유지한다.

1.3. 지원 플랫폼

- macOS 12 이상 (Apple Silicon / Intel)
- Windows 10/11 (x64)
- Linux (X11 / Wayland, glibc 2.31 이상)

1.4. 대상 사용자

렌더링 엔지니어, 그래픽 연구자, VFX 아티스트, 품질 관리 엔지니어.

2. 기술 스택

2.1. 선택 기준

기술 선택의 우선순위: ① 크로스 플랫폼 호환성, ② 경량 & 빠른 빌드, ③ 활발한 유지보수, ④ 라이선스 호환성 (MIT / BSD / Apache 2.0).

2.2. 스택 요약

항목	선택	이유
GUI 프레임워크	SDL3 + Dear ImGui (docking branch)	경량, 멀티 뷰포트, 키보드 우선, 크로스 플랫폼. ImGui docking으로 패널 레이아웃 자유로움.
이미지 로딩	stb_image (header-only)	외부 의존성 없음. PNG / JPEG / BMP / TGA / HDR / PIC 지원. 단일 헤더 포함.
렌더링 백엔드	OpenGL 3.3 Core	macOS / Windows / Linux 공통 지원. Vulkan 대비 코드 복잡성 낮음.
GL 로더	glad2	모던하고 경량. CMake FetchContent 통합 용이.
색상 관리	lcms2	ICC 프로파일 업계 표준 구현. 이미 시스템에 설치됨.
빌드 시스템	CMake 3.24+ FetchContent	의존성 자동 fetch. 별도 패키지 관리자 불필요.
C++ 표준	C++20	<code>std::jthread</code> , <code>concepts</code> , <code>ranges</code> 활용. MSVC / Clang / GCC 모두 지원.
GPU diff	GLSL fragment shader	제로 레이턴시 픽셀 diff. GPU에서 실시간 계산.
CPU diff	커스텀 SSIM (async)	백그라운드 스레드에서 지각적 유사도 계산. UI 블로킹 없음.
멀티 모니터	ImGui ViewportsEnable	OS 네이티브 윈도우 생성. 모니터 간 자유로운 이동.

2.3. 의존성 상세

2.3.1. SDL3

SDL3은 2024년에 정식 릴리즈된 SDL의 다음 세대이다. `SDL_CreateWindow`, `SDL_GL_CreateContext` 등 OpenGL 컨텍스트 관리를 담당한다. 이벤트 루프, 파일 다이얼로그 (`SDL_ShowOpenFileDialog`), 클립보드, 풀스크린 전환 등 플랫폼 추상화 레이어로 기능한다.

2.3.2. Dear ImGui (docking branch)

`imgui.h` / `imgui.cpp` + SDL3 backend + OpenGL3 backend를 사용한다. Docking branch는 윈도우 도킹과 멀티 뷰포트 (OS 네이티브 윈도우)를 지원한다.

2.3.3. stb_image / stb_image_write

단일 헤더 파일로 PNG, JPEG, BMP, TGA, HDR, PNM 포맷을 로딩한다. PNG 저장은 `stb_image_write`를 사용한다.

2.3.4. lcms2

ICC 색 프로파일 파싱 및 색 변환을 처리한다. sRGB, Display P3, AdobeRGB, ACES 등 다양한 색 공간 간 변환을 지원한다.

3. 아키텍처

3.1. 디렉토리 구조

```

av/
├── CMakeLists.txt          # 최상위 빌드 파일
├── src/
│   ├── main.cpp           # 진입점, 이벤트 루프
│   ├── app.{h,cpp}        # 애플리케이션 상태 및 움직임
│   ├── image_loader.{h,cpp} # 이미지 로딩 및 캐싱
│   ├── gl_texture.{h,cpp} # OpenGL 텍스처 관리
│   ├── diff_engine.{h,cpp} # diff 계산 (CPU/GPU)
│   ├── viewport.{h,cpp}   # 뷰포트 변환 (pan/zoom)
│   ├── ui/
│   │   ├── main_window.{h,cpp} # 메인 ImGui 레이아웃
│   │   ├── image_panel.{h,cpp} # 이미지 표시 패널
│   │   ├── compare_panel.{h,cpp} # 비교 모드 패널
│   │   ├── histogram.{h,cpp}  # 히스토그램 뷰
│   │   └── statusbar.{h,cpp}  # 상태 표시줄
│   └── shaders/
│       ├── image.vert      # 기본 이미지 vertex shader
│       ├── image.frag      # 이미지 표시 fragment shader
│       └── diff.frag       # diff 계산 fragment shader
├── third_party/            # FetchContent 다운로드 경로
│   ├── imgui/
│   ├── SDL3/
│   ├── stb/
│   └── glad/
└── test/
    ├── test_image_loader.cpp
    ├── test_diff_engine.cpp
    └── test_viewport.cpp

```

3.2. 모듈 의존성 다이어그램

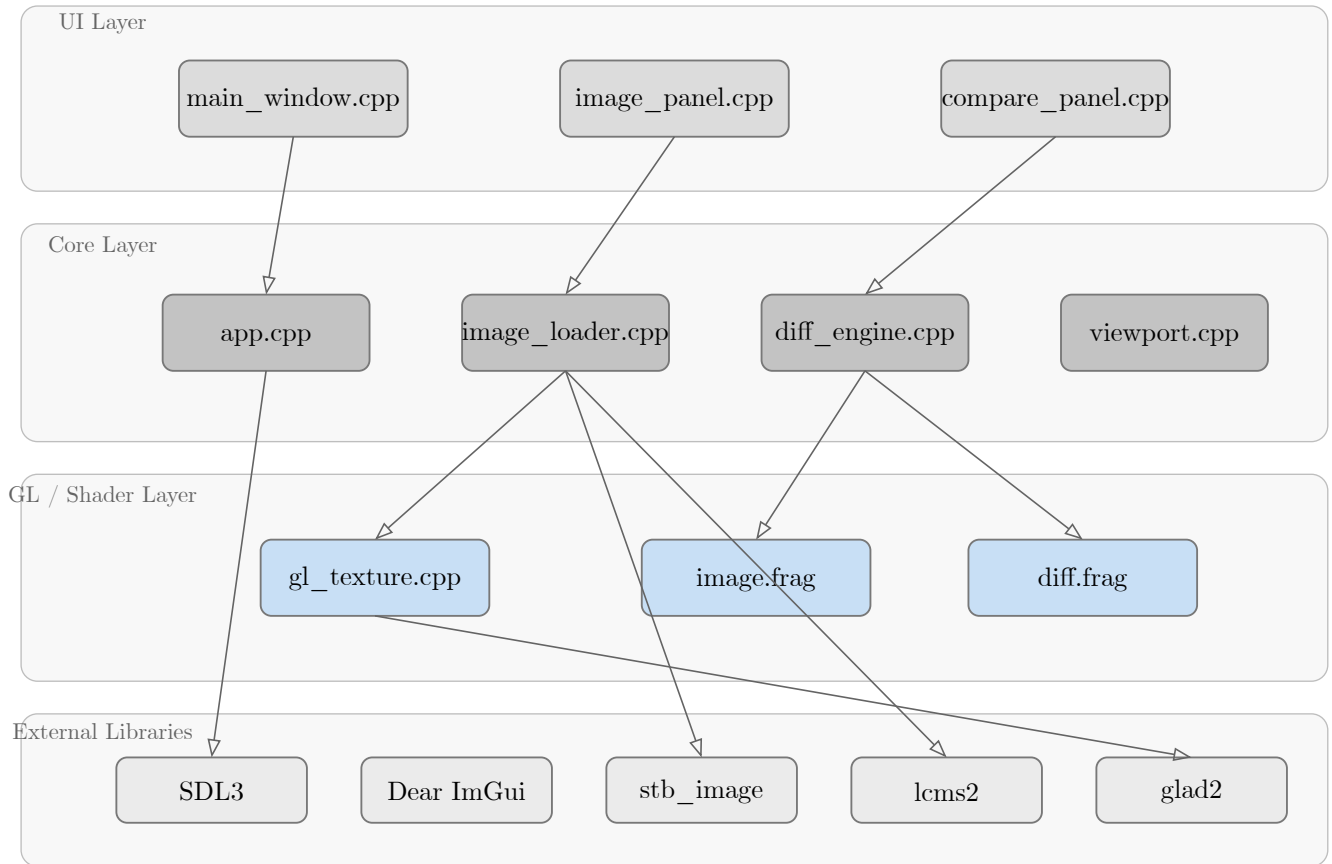


그림 1: 모듈 의존성 다이어그램

3.3. 데이터 흐름

이미지 로딩부터 화면 표시까지의 흐름:

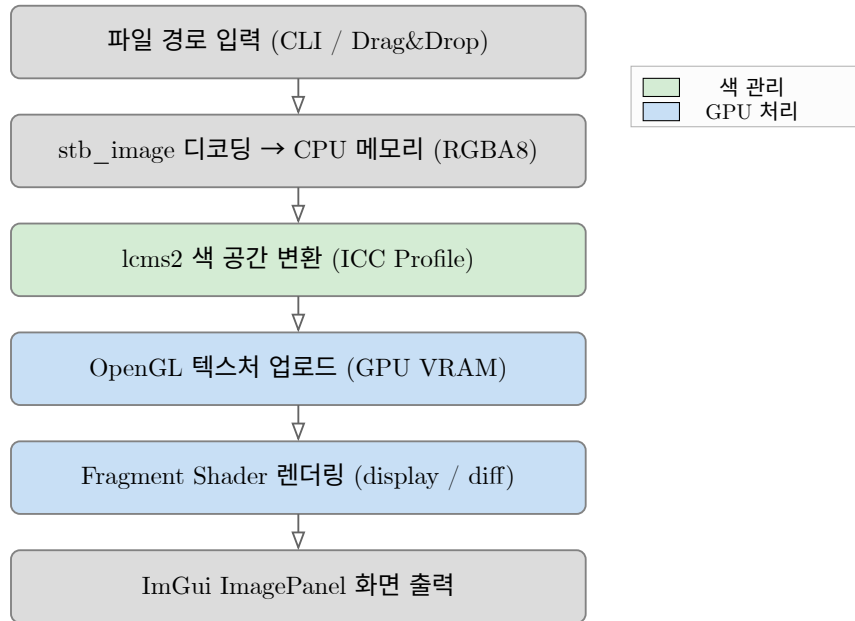


그림 2: 이미지 로딩 및 렌더링 데이터 흐름

3.4. 애플리케이션 상태 구조

```

// app.h (핵심 상태 구조체)
struct ImageEntry {
    std::string      path;
    std::vector<uint8_t> pixels;    // CPU 사이트 RGBA8
    int              width, height, channels;
    GLuint           texture_id;    // GPU 텍스처
    bool             loaded = false;
};

struct ViewportState {
    float zoom      = 1.0f;    // 배율 (0.1 ~ 32.0)
    float pan_x     = 0.0f;    // 픽셀 단위 오프셋
    float pan_y     = 0.0f;
    bool fit        = true;    // fit-to-window 모드
};

struct DiffState {
    enum class Mode { None, PixelAbsolute, PixelRelative, SSIM };
    Mode mode       = Mode::None;
    float amplify   = 1.0f;    // diff 강조 배율
    float ssim_score = 0.0f;    // 비동기 계산 결과
};

```

```
struct AppState {  
    std::array<ImageEntry, 2> images;    // [0]: 좌측, [1]: 우측  
    std::array<ViewportState, 2> views;  
    DiffState diff;  
    bool sync_viewports = true;    // 뷰포트 동기화  
    bool show_histogram = false;  
};
```

4. 핵심 기능 사양

4.1. 이미지 로딩

4.1.1. 지원 포맷

포맷	설명	비고
PNG	8/16-bit, alpha 지원	최우선 지원
JPEG / JPG	표준 손실 압축	EXIF 방향 자동 적용
BMP	비트맵	Windows 호환
TGA	Targa	알파 지원
HDR	Radiance RGBE	float 텍스처로 업로드
PIC	Softimage PIC	-
PNM / PBM / PGM / PPM	Netpbm 포맷	-

4.1.2. 캐싱 정책

- 최근 사용 이미지 최대 8개를 메모리에 캐시한다.
- GPU 텍스처는 OpenGL 오브젝트로 유지하며, 교체 시 `glDeleteTextures` 로 해제한다.
- 파일 크기 4096 x 4096 이상의 이미지는 비동기(`std::jthread`) 로딩을 적용한다.
- 로딩 중 진행 상황은 상태 표시줄에 표시한다.

4.1.3. 색 공간 처리

1. `stb_image` 디코딩 → RGBA8 (또는 RGBA32F for HDR)
2. 파일에 ICC 프로파일의 있으면 `lcms2`로 추출
3. `lcms2` 변환: 파일 색공간 → sRGB (display)
4. OpenGL 텍스처: GL_SRGB8_ALPHA8 (SDR) / GL_RGBA32F (HDR)
5. Fragment shader에서 gamma correction 적용

4.2. 뷰포트 관리

4.2.1. 확대/축소 (Zoom)

- 배율 범위: 0.1x ~ 32x

- 마우스 휠 또는 **+**/**-** 키로 단계별 확대/축소
- 줌 레벨: 12.5%, 17%, 25%, 33%, 50%, 67%, 100%, 150%, 200%, 300%, 400%, 800%, 1600%, 3200%
- **0** 키: **Fit to Window** (윈도우 크기에 맞게 자동 조절)
- **1** 키: **100% (1:1 픽셀)** 모드

4.2.2. 패닝 (Pan)

- 마우스 좌측 드래그 또는 방향키(**H**/**J**/**K**/**L**) 로 이동
- **Shift** + 방향키: 빠른 이동 (10픽셀 단위)
- 이미지 경계에서 자동 클램핑 (선택 가능)

4.2.3. 뷰포트 동기화

비교 모드에서 두 뷰포트를 동기화한다:

- **s** 키로 동기화 토글
- 동기화 시 한쪽 패닝/줌이 반대쪽에 즉시 반영됨
- 다른 해상도 이미지: 픽셀 기준 또는 비율 기준 선택 가능

4.3. 이미지 비교 모드

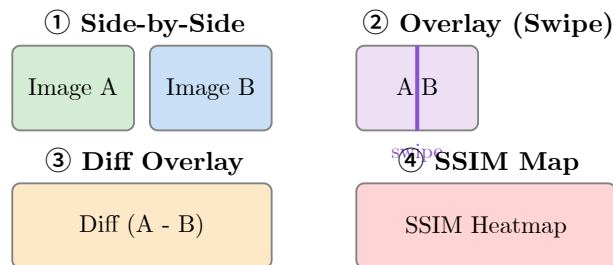


그림 3: 이미지 비교 모드 개요

4.3.1. Side-by-Side 모드

두 이미지를 좌우로 나란히 표시한다. 기본 비교 모드이며, **Ctrl+1** 로 전환한다. 뷰포트 동기화(**s**)를 통해 양쪽이 동시에 패닝/줌된다.

4.3.2. Overlay (Swipe) 모드

하나의 패널에서 마우스 위치 기준으로 좌측은 Image A, 우측은 Image B를 표시한다. **Ctrl+2** 로 전환. 슬라이더 위치는 마우스 이동으로 조작하며, 마우스를 고정하면 위치가 잠긴다.

4.3.3. Diff 모드

GPU fragment shader에서 실시간으로 픽셀 차이를 계산하여 시각화한다. **Ctrl+3** 으로 전환.

4.4. Diff 엔진

4.4.1. GPU Diff (실시간)

Fragment shader에서 두 텍스처 샘플링 후 차이를 계산한다:

```
// diff.frag (핵심 코직)
uniform sampler2D texA;
uniform sampler2D texB;
uniform int    diff_mode;    // 0=Abs, 1=Relative, 2=Component
uniform float amplify;      // 강조 배율

void main() {
    vec4 a = texture(texA, uv);
    vec4 b = texture(texB, uv);
    vec4 diff = abs(a - b);

    if (diff_mode == 1) {      // Relative
        diff = diff / (a + 0.001);
    }
    fragColor = vec4(diff.rgb * amplify, 1.0);
}
```

4.4.2. CPU Diff (SSIM)

Structural Similarity Index Measure를 비동기로 계산한다:

- `std::jthread` 로 백그라운드 실행 (UI 블로킹 없음)
- 11x11 Gaussian 윈도우 기반 SSIM
- 결과는 히트맵 텍스처로 GPU에 업로드
- 계산 완료 시 상태 표시줄에 SSIM 점수 표시 (0.0 ~ 1.0)

4.4.3. Diff 모드 유형

모드	설명	단축키
Pixel Absolute	$ A - B $ 절댓값 차이. 흰색 = 최대 차이.	Ctrl+3
Pixel Relative	$ A - B / A$. 밝기 보정된 차이.	Ctrl+4
False Color	차이 강도를 heatmap (파랑→빨강)으로 표시.	Ctrl+5
SSIM Map	지각적 유사도 맵. 비동기 CPU 계산.	Ctrl+6

4.5. 멀티 모니터 지원

ImGui `ViewportsEnable` 플래그를 사용하여 OS 네이티브 윈도우를 생성한다.

- 패널을 드래그하여 다른 모니터로 이동 가능
- 각 OS 윈도우는 독립적인 OpenGL 컨텍스트 공유
- HiDPI (Retina) 디스플레이 자동 감지 및 스케일 적용
- `Alt+Enter`: 현재 패널 풀스크린 토글

4.6. 히스토그램

- R / G / B / A / Luminance 채널 히스토그램 표시
- 로그 스케일 토글 (H 키)
- 두 이미지 히스토그램 오버레이 지원
- 뷰포트 영역 기준 히스토그램 업데이트 (줌 영역 반영)

5. 키보드 바인딩

5.1. 파일 조작

키	동작
O	파일 열기 다이얼로그 (Image A)
Shift+O	파일 열기 다이얼로그 (Image B)
Ctrl+O	최근 파일 목록 열기
Ctrl+W	현재 이미지 닫기
Ctrl+R	현재 이미지 재로딩 (파일 변경 감지 후 수동 갱신)
Ctrl+S	diff 결과 이미지를 PNG로 저장
Ctrl+C	현재 뷰포트 영역을 클립보드로 복사
Q / Ctrl+Q	애플리케이션 종료

5.2. 뷰포트 조작

키	동작
O	Fit to Window (창 크기에 맞춤)
1	1:1 픽셀 (100% 배율)
2	200% 배율
4	400% 배율
+ / =	확대 (다음 줌 레벨)
-	축소 (이전 줌 레벨)
H / ←	좌측 패닝
L / →	우측 패닝
K / ↑	위 패닝
J / ↓	아래 패닝
Shift+H/L/K/J	빠른 패닝 (10픽셀 단위)
G	이미지 가운데로 이동
F	Fit to Window 토글

5.3. 비교 모드 전환

키	동작
Ctrl+1	Side-by-Side 모드
Ctrl+2	Overlay (Swipe) 모드
Ctrl+3	Diff: Pixel Absolute 모드
Ctrl+4	Diff: Pixel Relative 모드
Ctrl+5	Diff: False Color 모드
Ctrl+6	SSIM Map 모드 (비동기 계산 시작)
S	뷰포트 동기화 토글
Tab	Image A / Image B 포커스 전환
Space	A/B 이미지 빠른 스왑 (토글)

5.4. Diff 파라미터

키	동작
[	Diff 강조 배율 감소 (amplify -0.5)
]	Diff 강조 배율 증가 (amplify +0.5)
\	Diff 강조 배율 초기화 (1.0)
Ctrl+D	Diff 모드 전체 비활성화

5.5. UI 패널 토글

키	동작
I	이미지 정보 패널 토글 (해상도, 색공간, 파일 크기)
Shift+H	히스토그램 패널 토글
P	픽셀 정보 표시 토글 (마우스 위치의 RGBA 값)
Alt+Enter	풀스크린 토글
Ctrl+,	환경설정 창 열기
? / F1	키보드 단축키 도움말

6. CLI 인터페이스

6.1. 기본 사용법

```
av [옵션] [이미지A] [이미지B]
```

단일 이미지 열기:

```
av image.png
av /path/to/render.exr
```

두 이미지 비교:

```
av before.png after.png
av --diff-mode=absolute render_A.png render_B.png
```

6.2. 옵션 목록

옵션	기본값	설명
<code>--diff-mode <mode></code>	<code>none</code>	<code>none</code> / <code>abs</code> / <code>rel</code> / <code>falsecolor</code> / <code>ssim</code>
<code>--zoom <factor></code>	<code>fit</code>	초기 줌 배율. <code>fit</code> , <code>1</code> , <code>2.0</code> 등
<code>--sync</code>	<code>true</code>	뷰포트 동기화 활성화 (기본 on)
<code>--no-sync</code>	-	뷰포트 동기화 비활성화
<code>--amplify <val></code>	<code>1.0</code>	Diff 강조 배율 (0.1 ~ 100.0)
<code>--fullscreen</code>	<code>false</code>	풀스크린으로 시작
<code>--geometry <WxH></code>	자동	초기 윈도우 크기 (예: <code>1920x1080</code>)
<code>--profile <file></code>	sRGB	ICC 색 프로파일 파일 경로
<code>--no-color-mgmt</code>	<code>false</code>	색 관리 비활성화 (원시 픽셀 표시)
<code>--version</code>	-	버전 정보 출력 후 종료
<code>-h</code> , <code>--help</code>	-	도움말 출력

6.3. 사용 예시

```
# 두 렌더링 결과를 absolute diff로 비교
av --diff-mode=abs --amplify=5 ref.png output.png

# SSIM 분석 모드로 시작
av --diff-mode=ssim before.png after.png

# 특정 ICC 프로파일로 비교
av --profile=DisplayP3.icc a.png b.png

# 2x 줌으로 시작, 동기화 비활성화
av --zoom=2 --no-sync imageA.jpg imageB.jpg
```

6.4. 종료 코드

코드	의미
0	정상 종료
1	파일 열기 실패 또는 잘못된 인수
2	초기화 오류 (SDL / OpenGL 실패)

7. 구현 로드맵

7.1. 단계별 계획

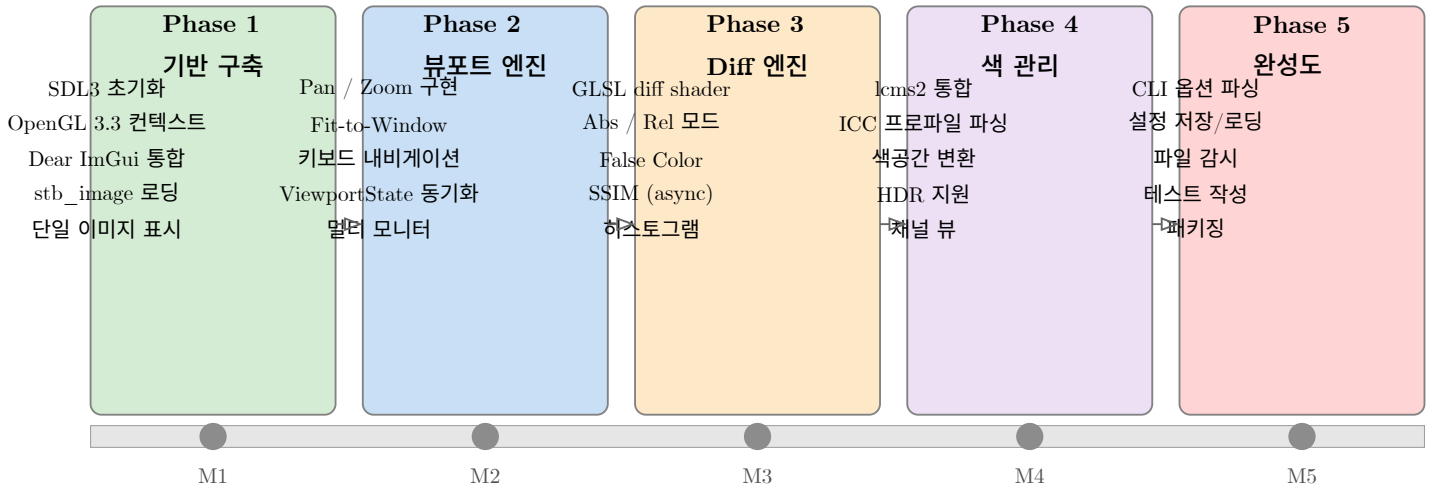


그림 4: 구현 단계 로드맵

7.2. 상세 마일스톤

단계	마일스톤	완료 기준
Phase 1	M1: 기반 구축	단일 이미지를 윈도우에 표시. ImGui UI 기본 레이아웃 동작.
Phase 2	M2: 뷰포트 엔진	pan/zoom/fit 완전 동작. 두 이미지 side-by-side 동기화 패닝.
Phase 3	M3: Diff 엔진	4가지 diff 모드 GPU 실시간 동작. SSIM 백그라운드 계산 및 히트맵 표시.
Phase 4	M4: 색 관리	ICC 프로파일 내장 PNG 정확한 색 재현. Display P3 / AdobeRGB 지원.
Phase 5	M5: 릴리즈 준비	CLI 완전 구현. 설정 파일 저장. 단위 테스트 커버리지 60% 이상. macOS / Windows / Linux 패키지 생성.

7.3. 빌드 환경 설정

7.3.1. 의존성 설치 (FetchContent)

```
# CMakeLists.txt (핵심 부분)
cmake_minimum_required(VERSION 3.24)
```

```

project(av CXX)
set(CMAKE_CXX_STANDARD 20)

include(FetchContent)

FetchContent_Declare(SDL3
  GIT_REPOSITORY https://github.com/libsdl-org/SDL.git
  GIT_TAG release-3.2.0)

FetchContent_Declare(imgui
  GIT_REPOSITORY https://github.com/ocornut/imgui.git
  GIT_TAG docking)

FetchContent_Declare(glad
  GIT_REPOSITORY https://github.com/Dav1dde/glad.git
  GIT_TAG glad2)

FetchContent_MakeAvailable(SDL3 imgui glad)

```

7.3.2. 빌드 방법

```

cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc)
./build/av

```

7.4. 테스트 전략

- 단위 테스트: `image_loader`, `diff_engine`, `viewport` 모듈 별 독립 테스트
- 골든 이미지 테스트: 알려진 diff 결과를 미리 저장하고 회귀 테스트
- 플랫폼 테스트: GitHub Actions에서 macOS / Ubuntu / Windows 매트릭스 빌드
- 수동 테스트: 다양한 이미지 포맷 및 ICC 프로파일 조합 확인

av Technical Specification v0.1

2026-02-27

이 문서는 구현 전 사양서이며, 구현 중 변경될 수 있습니다.